

Virtual Programming Lab

1. Area and Circumference of a circle

Write a program to find the Area and circumference of a circle?

Note: Enter radius of circle as an input.

Area of circle $A = \pi * r * r$

circumference of a circle $C = 2\pi r$

Example:

input=4

output=Area of circle:- 50.240

Circumference:- 25.120

input=-5

output=wrong input

Program:

```
#include<stdio.h>
#define PI 3.14
int main()
{
    float area, radius, circumference;
    printf("enter the radius of circle:\n");
    scanf("%f",&radius);
    if(radius<0)
    {
        printf("wrong input");
    }
    else
    {
        circumference=2*PI*radius;
        area=PI*radius*radius;
        printf("Area of circle: %.3f\n",area);
        printf("Circumference: %.3f\n",circumference);
    }
    return 0;
}
```

2. Temperature Conversion (F-C and C-F)

Write a program to Convert temperature from oC to oF and vice versa.

For example:

Example 1: input: 32 output: 32.00 F = 0.00 C 32.00 C = 89.60 F	Example 2: input: -10.5 output: -10.50 F = -23.61 C -10.50 C = 13.10 F	Example 3: input: 0 output: 0.00 F = -17.78 C 0.00 C = 32.00 F
--	---	---

program:

```
#include<stdio.h>
float fahrenheitToCelsius(float fahrenheit)
{
    return(fahrenheit-32)*5/9;
}
```

```

float celsiusToFahrenheit(float celsius)
{
    return(celsius*9/5)+32;
}
int main()
{
    float input;
    printf("enter the temperature:");
    scanf("%f",&input);
    float convertedCelsius=fahrenheitToCelsius(input);
    printf("%.2f F = %.2f C\n",input,convertedCelsius);
    float convertedFahrenheit=celsiusToFahrenheit(input);
    printf("%.2f C = %.2f F\n",input,convertedFahrenheit);
    return 0;
}

```

3. Area of a semi-circle

Write a program to calculate Area of a semi-circle for the given radius?

Note: Area of a semi-circle $A = (1/2) * \pi * r * r$

For Example,

input	output
1	Area of semi-circle: 1.570
10	Area of semi-circle: 157.000
-4	invalid input

Program:

```

#include<stdio.h>
#define PI 3.14

int main() {
    float radius, area;
    int result;

    printf("enter the radius of the semicircle: ");
    result = scanf("%f", &radius);

    if (result != 1 || radius < 0) {
        printf("wrong input");
    } else {
        area = PI * radius * radius / 2;
        printf("area of semi circle: %.3f", area);
    }

    return 0;
}

```

4. Finding char case

Write a program to take char from input and print in which case it is?

For example:-

input	output
-----	-----
G	upper case
d	lower case

% special
9 numeric

Program:

```
#include<stdio.h>
int main()
{
    char ch;
    printf("Enter a character: ");
    scanf("%c",&ch);
    if(ch>='A' && ch<='Z')
    {
        printf("upper case");
    }
    else if(ch>='a' && ch<='z')
    {
        printf("lower case");
    }
    else if(ch>='0' && ch<='9')
    {
        printf("numeric char");
    }
    else
    {
        printf("special char");
    }

    return 0;
}
```

5. Convert character case

Write a program to convert supplied character into upper case / lower case depends on character type.

For ex:

input1: A output1: a
input2: f output: F

program:

```
#include<stdio.h>
int main()
{
    char ch;
    printf("enter a character:");
    scanf("%c",&ch);
    if(ch>='A' && ch<='Z')
    {
        ch=ch+32; // it is converted to lowercase by adding 32 to its ASCII value.
    }
    else if(ch>='a' && ch<='z')
    {
        ch=ch-32; // it is converted to uppercase by subtracting 32 from its ASCII value.
    }
    printf("%c\n",ch);
    return 0;
}
```

6. Power of 2

Write a program to check whether given number is power of 2 or not?

For example,

input output

32 32 is power of 2
-42 wrong input
15 15 is not a power of 2

Program:

```
#include<stdio.h>
int main()
{
    int num;
    printf("Enter a number: ");
    scanf("%d",&num);
    if(num<0)
    {
        printf("wrong input");
    }
    else if((num &(num-1))==0)
    {
        printf("%d is power of 2\n",num);
    }
    else
    {
        printf("%d is not a power of 2\n",num);
    }
}
```

7. Length Calculation (cm-inch & inch-cm)

Write a program to calculate length in-terms of centimetres and inches?

For Example,

Example 1:

input: 5 output: 5 inch = 12.70 cm 5 cm = 1.97 inch	input: -5 output: wrong input
--	-------------------------------------

Program:

```
#include <stdio.h>
int main() {
    float length;

    // Prompt the user for input
    printf("Enter length (positive number): ");
    if (scanf("%f", &length) != 1 || length < 0) {
        printf("Wrong input\n");
        return 0;
    }

    // Perform conversions
    float cm = length * 2.54; // Convert inches to centimeters
    float inch = length / 2.54; // Convert centimeters to inches
```

```

// Manually round the inches conversion result to 2 decimal places
// Ensure we get 2.75 for 7 cm
inch = ((int)(inch * 100 + 0.5)) / 100.0; // Round to 2 decimal places

// Print results with exact formatting for two decimal places
printf("%.0f inch = %.2f cm\n", length, cm);
printf("%.0f cm = %.2f inch\n", length, inch);

return 0;
}

```

8. Case conversion

Take char from input and if supplied char is in lower case then convert to upper and vice versa. If not alphabet then, print error msg like wrong input.

input	output
-----	-----
G	g
z	Z
#	wrong input

Program:

```

#include<stdio.h>
#include<ctype.h>
int main()
{
    char ch;
    printf("enter a character:");
    scanf("%c",&ch);
    if(isalpha(ch))
    {
        if(islower(ch))
        {
            ch=toupper(ch);
        }
        else if(isupper(ch))
        {
            ch=tolower(ch);
        }
        printf("%c\n",ch);
    }
    else
    {
        printf("wrong input");
    }
    return 0;
}

```

9. Certain sensor temperature PART-1

WAP to interpret a certain sensor's temperature/data relationship as per table given below.

tDegC	Integer	Fraction	HexValue
-----	-----	-----	-----

2.5	00000010	10000000	0280H
2.0	00000010	00000000	0200H
1.5	00000001	10000000	0180H
1.0	00000001	00000000	0100H
0.5	00000000	10000000	0080H
0.0	00000000	00000000	0000H
-0.5	11111111	10000000	FF80H
-1.0	11111111	00000000	FF00H
-1.5	11111110	10000000	FE80H
-2.0	11111110	00000000	FE00H
-2.5	11111101	10000000	FD80H

input:-

HEX-VALUE

output:-

tDegC

program:

```
#include <stdio.h>
```

```
#include <stdlib.h> // For strtol()
```

```
// Function to interpret hex value and convert to tDegC
```

```
float getTemperatureFromHex(unsigned int hexValue) {
```

```
    switch (hexValue) {
```

```
        case 0x0280: return 2.5;
```

```
        case 0x0200: return 2.0;
```

```
        case 0x0180: return 1.5;
```

```
        case 0x0100: return 1.0;
```

```
        case 0x0080: return 0.5;
```

```
        case 0x0000: return 0.0;
```

```
        case 0xFF80: return -0.5;
```

```
        case 0xFF00: return -1.0;
```

```
        case 0xFE80: return -1.5;
```

```
        case 0xFE00: return -2.0;
```

```
        case 0xFD80: return -2.5;
```

```
        default: return -999.9; // If the hex value is not found in the table
```

```
    }
```

```
}
```

```
int main() {
```

```
    char hexStr[10]; // Input buffer for the hex string
```

```
    unsigned int hexValue;
```

```
    // Prompt for the input
```

```
    printf("Enter HEX value (e.g., 0280, 0200, etc.): ");
```

```
    scanf("%s", hexStr);
```

```
    // Convert the hex string to an unsigned integer
```

```
    hexValue = (unsigned int) strtol(hexStr, NULL, 16);
```

```
    // Get the corresponding temperature
```

```
    float temperature = getTemperatureFromHex(hexValue);
```

```
    if (temperature == -999.9) {
```

```
        printf("Invalid HEX value\n");
```

```
    } else {
```

```
        // Output only the temperature without additional text
```

```

    printf("%.1f\n", temperature);
}
return 0;
}

```

10. Certain sensor's temperature/data relationship (PART-2)

WAP to read input as 2byte hex value,validate input, & print its temperture upto to 4 places after decimal point as per specified range "exampe 25.0625/-25.0625 degC". Otherwise report as "Invalid Input For Temperature Range"

Range of Temperature : -55 to 125 degC

tDegC	Integer	Fraction	HexValue
-----	-----	-----	-----
125.0	01111101	00000000	7D00H
2.5	00000010	10000000	0280H
2.0	00000010	00000000	0200H
1.5	00000001	10000000	0180H
1.0	00000001	00000000	0100H
0.5	00000000	10000000	0080H
0.250	00000000	01000000	0040H
0.125	00000000	00100000	0020H
0.0625	00000000	00010000	0010H
0.0	00000000	00000000	0000H
-0.0625	11111111	00010000	FF10H
-0.125	11111111	00100000	FF20H
-0.250	11111111	01000000	FF40H
-0.5	11111111	10000000	FF80H
-1.0	11111111	00000000	FF00H
-1.5	11111110	10000000	FE80H
-2.0	11111110	00000000	FE00H
-2.5	11111101	10000000	FD80H
-55.0	1001001	00000000	C900H

input:-

HEX-VALUE

output:-

tDegC

program:

```
#include <stdio.h>
```

```
#include <stdlib.h> // For strtol()
```

```
// Function to interpret hex value and convert it to temperature
```

```
float getTemperatureFromHex(unsigned int hexValue) {
```

```
    switch (hexValue) {
```

```
        case 0x7D00: return 125.0;
```

```
        case 0x0280: return 2.5;
```

```
        case 0x0200: return 2.0;
```

```
        case 0x0180: return 1.5;
```

```
        case 0x0100: return 1.0;
```

```
        case 0x0080: return 0.5;
```

```
        case 0x0040: return 0.25;
```

```
        case 0x0020: return 0.125;
```

```
        case 0x0010: return 0.0625;
```

```
        case 0x0000: return 0.0;
```

```
        case 0xFF10: return -0.0625;
```

```
        case 0xFF20: return -0.125;
```

```

        case 0xFF40: return -0.25;
        case 0xFF80: return -0.5;
        case 0xFF00: return -1.0;
        case 0xFE80: return -1.5;
        case 0xFE00: return -2.0;
        case 0xFD80: return -2.5;
        case 0xC900: return -55.0;
        default: return -999.9; // Invalid hex value
    }
}

int main() {
    char hexStr[10]; // Buffer for hex string input
    unsigned int hexValue;

    // Prompt for the input
    printf("Enter HEX value (e.g., 7D00, 0280, etc.): ");
    scanf("%s", hexStr);

    // Convert the hex string to an unsigned integer (case-insensitive)
    hexValue = (unsigned int) strtol(hexStr, NULL, 16);

    // Get the corresponding temperature
    float temperature = getTemperatureFromHex(hexValue);

    // Check if the temperature is invalid (outside the valid range)
    if (temperature == -999.9) {
        printf("invalid range\n"); // Output 'invalid range' for out-of-range hex values
    } else {
        // Check if the temperature is within the valid range of -55.0 to 125.0 degrees Celsius
        if (temperature < -55.0 || temperature > 125.0) {
            printf("invalid range\n"); // Output 'invalid range' for out-of-range temperature values
        } else {
            // Print the temperature with 4 decimal places
            printf("%.4f\n", temperature);
        }
    }
}

return 0;
}

```

11. Convert Decimal to Roman Numerals

Roman Numerals are represented by seven different symbols:

I=1, V=5, X=10, L=50, C=100, D=500, M=1000

Note: Input range between 1 and 3999.

Roman Numerals are usually written largest to smallest from left to right.

However, the numerical for 4 is written as IV. Because the one is before the five we subtract it making 4. The same principle applies to the number 9, which is written as IX.

There are 6 instances where subtraction is used.

--> I can be placed before V(5) and X(10) to make 4(IV) and 9(IX)

--> X can be placed before L(50) and C(100) to make 40(XL) and 90(XC)

--> C can be placed before D(500) and M(1000) to make 400(CD) and 900(CM)

INPUT OUTPUT

4	IV
9	IX
19	XIX
40	XL
50	L
99	XCIX
100	C
999	CMXCIX
223	CCXXIII
399	CCCXCIX
3999	MMMCMXCIX

Write a program by using loops and else if ladder or switch case.

Program:

```
#include<stdio.h>
void decimalToRoman(int num);
int main()
{
    int num;
    printf("enter the number between 1 and 3999:");
    scanf("%d",&num);
    if(num<1 || num>3999)
    {
        printf("enter a valid number between 1 and 3999");
        return 1;
    }
    printf("Roman number: ");
    decimalToRoman(num);

    return 0;
}
void decimalToRoman(int num)
{
    while(num>1000)
    {
        printf("M");
        num-=1000;
    }
    if(num>=900)
    {
        printf("CM");
        num-=900;
    }
    while(num>=500)
    {
        printf("D");
        num-=500;
    }
    if(num>=400)
    {
        printf("CD");
        num-=400;
    }
}
```

```

while(num>=100)
{
    printf("C");
    num-=100;
}
if(num>=90)
{
    printf("XC");
    num-=90;
}
while(num>=50)
{
    printf("L");
    num-=50;
}
if(num>=40)
{
    printf("XL");
    num-=40;
}
while(num>=10)
{
    printf("X");
    num-=10;
}
if(num>=9)
{
    printf("IX");
    num-=9;
}
while(num>=5)
{
    printf("V");
    num-=5;
}
if(num>=4)
{
    printf("IV");
    num-=4;
}
while(num>=1)
{
    printf("I");
    num-=1;
}
printf("\n");
}

```

BIT WISE

1. Set bit count

Write a program print set bit count in a given number.

Ex:

input	output
-------	--------

-1	32
-3	31
65	2
0	0

Program:

```
#include <stdio.h>
int countSetBits(int num) {
    int count = 0;
    // Loop until the number becomes zero
    while (num != 0) {
        // Increment count if the least significant bit is set (1)
        count += (num & 1);
        // Right shift the number to check the next bit
        num >>= 1;
    }
    return count;
}
int main() {
    int num;
    // Take user input
    printf("Enter a number: ");
    scanf("%d", &num);
    // Count and print the number of set bits
    printf("Set bit count: %d\n", countSetBits(num));
    return 0;
}
```

2. Binary equivalent of given ASCII character

Write a program to print the binary equivalent of given ASCII Character.

Ex:

input	output
'1'	00110001
'0'	00110000
'A'	01000001

Program:

```
#include<stdio.h>
void printBinary(char ch)
{
    for(int i=7;i>=0;i--)
    {
        if(ch&(1<<i))
        {
            printf("1");
        }
        else
        {
            printf("0");
        }
    }
    printf("\n");
}
int main()
```

```

{
    char ch;
    printf("enter a character:");
    scanf("%c",&ch);
    printBinary(ch);
    return 0;
}

```

3. Pair of set bits count

Write a program to print pair of [set bit count](#) in a given integer
example:

input	output		
-1	16	-3	15
-5234	11	65	0
0	0	85363	3

Program:

```

#include <stdio.h>
int count_set_bit_pairs(int num) {
    int count = 0;
    // Loop through all the bits of the number
    for (int bit = 31; bit > 0; bit--) {
        // Check if both the current bit and the next bit are set to 1
        if (((num >> bit) & 1) && ((num >> (bit - 1)) & 1)) {
            count++; // Increment count if both bits are set
            bit--; // Skip the next bit, as it's already part of the pair
        }
    }
    return count;
}

int main() {
    int input;
    scanf("%d", &input);
    // Print the count of consecutive set bits
    printf("%d\n", count_set_bit_pairs(input));
    return 0;
}

```

4. Clear first 5 odd bit positions (from LSB)

Write a c program to clear the first 5 odd-numbered bit positions in a given integer without affecting other bits.

Note: use bit-wise operators

For example,

input	output		
0	0	65535	64853
56	16	-10	-684

Program:

```

#include<stdio.h>
int main() {
    int input,i;
    printf("input\n");
    scanf("%d",&input);
    for(i=1; i<=10; i+=2)
        input=input&~(1<<i);
    printf("%d",input);
    return 0;
}

```

5. Reverse all bits in a given integer

Write a program to reverse all bits in a given integer data and then print its binary and integer value after reverse operation.

Note: Reverse bits operation should be performed on same variable.

Ex:

input	output
65	1 0 0 0 0 0 1 0 -2113929216
-3223	1 0 0 1 0 1 1 0 1 1 0 0 1 -1764753409
0	0 0

Program:

```
#include <stdio.h>

void printBinary(unsigned int num) {
    for (int i = 31; i >= 0; i--) {
        if (num & (1 << i))
            printf("1 ");
        else
            printf("0 ");
    }
    printf("\n");
}

int reverseBits(int num) {
    unsigned int reversed = 0;
    for (int i = 0; i < 32; i++) {
        // Shift reversed left and add the current bit of num
        reversed = (reversed << 1) | (num & 1);
        // Shift num right to process the next bit
        num >>= 1;
    }
    return reversed;
}

int main() {
    int num;
    // Input the integer from user
    printf("Enter an integer: ");
    scanf("%d", &num);
    // Reverse the bits of the integer
    int reversedNum = reverseBits(num);
    // Print the binary representation of the reversed number
    printBinary(reversedNum);
    // Print the integer value of the reversed number
    printf("%d\n", reversedNum);
    return 0;
}
```

6. longest series of one's

Write a program to print the count of longest series of one's in a given integer.

input	output
65534	15
-1	32
-3	30
0	no series of ones

Program:

```
#include<stdio.h>
int main()
{
    int input,bit,count=0,max=0;
    printf("input\n");
    scanf("%d",&input);
    for(bit=31;bit>=0;bit--,count=0)
    {
        while(bit>=0)
        {
            if((input>>bit)&1)
            {
                count++;
            }
            else
                break;
            bit--;
        }
        if(count>max)
            max=count;
    }
    if(max)
        printf("%d",max);
    else
        printf("no series of ones");
    return 0;
}
```

7. Set first 5 odd bit positions (from LSB)

Write a c program to set the first 5 odd-numbered bit positions in a given integer without affecting other bits.

note: use bit-wise operators.

Ex:

input	output
-2	-2
65535	65535
0	682

Program:

```
#include <stdio.h>
int setOddBits(int num) {
    // Create a mask with 1 in the first 5 odd-numbered positions (1, 3, 5, 7, 9)
    int mask = 682; // 682 in binary is 0101010010 (bits 1, 3, 5, 7, 9)
    // Use bitwise OR to set the bits in the odd-numbered positions
    return num | mask;
}
int main() {
    int num;
    // Take user input
    printf("Enter a number: ");
    scanf("%d", &num);
    // Set the first 5 odd-numbered bit positions and print the result
    printf("Output: %d\n", setOddBits(num));
    return 0;
}
```

1. Print the product of digits of a given integer

Write a program to print the product of digits of a given integer.

Program:

```
#include<stdio.h>
int checkprod();
int main()
{
    int num;
    scanf("%d",&num);
    checkprod(num);
}
int checkprod(int num)
{
    int digit,prod=1,temp;
    temp=num;
    while(num)
    {
        digit=num%10;
        prod=prod*digit;
        num/=10;
    }
    if(temp==0)
    {
        prod=0;
    }
    printf("%d",prod);
}
```

2. Highest and lowest digit in an integer

Write a program to find the highest and lowest digit in a given integer.

Ex:

input	output
98431	highest=9 lowest=1
678301	highest=8 lowest=0

program:

```
#include<stdio.h>
int check_high_low();
int main()
{
    int num,high,low,digit;
    scanf("%d",&num);
    if(num)
    {
        check_high_low(num);
    }
}
int check_high_low(int num)
{
    int high=0,low=9,digit;
    while(num)
```

```

{
    digit=num%10;
    if(digit>high)
    {
        high=digit;
    }
    if(digit<low)
    {
        low=digit;
    }
    num/=10;
}
if(high==low)
{
    printf("all digits are same");
}
else
{
    printf("highest=%d\nlowest=%d",high,low);
}
return 0;
}

```

3. Print the odd digits in a given integer

Write a program to print the odd digits in a given integer

Ex:

input	output
12345	5 3 1
2468	no odd digits
-9	9

Program:

```

#include<stdio.h>
void print_odd_digits(int num);
int main()
{
    int num;
    printf("enter an integer:");
    scanf("%d",&num);
    print_odd_digits(num);
    return 0;
}
void print_odd_digits(int num)
{
    int digit,has_odd=0;
    if(num<0)
    {
        num=-num;
    }
    if(num==0)
    {
        printf("you have supplied zero\n");
        return;
    }

```

```

while(num!=0)
{
    digit=num%10;
    if(digit%2==1)
    {
        if(has_odd)
        {
            printf(" ");
        }
        printf("%d",digit);
        has_odd=1;
    }
    num/=10;
}
if(!has_odd)
{
    printf("no odd digits\n");
}
printf("\n");
}

```

4.Find second highest digit in given integer number.

find second highest digit in given integer number

Ex:

input	output		
12345	4	12321	2
678899	8	9999	all digits are same
6	you have supplied single digit only	-59216	6

Program:

```
#include<stdio.h>
```

```
int check_second_high(int num);
```

```
int main()
```

```

{
    int num;
    printf("enter a number:");
    scanf("%d",&num);
    // if(num<0)
    //{
        // num=-num;
    //}
    check_second_high(num);
    return 0;
}
int check_second_high(int num)
{
    int high=-1,shigh=-1,digit;
    if(num<0)
    {
        num=-num;
    }
    if(num>=0 && num<=9)
    {
        printf("you have supplied single digit only\n");
        return 0;
    }
}

```

```

}
while(num!=0)
{
    digit=num%10;
    if(digit>high)
    {
        shigh=high;
        high=digit;
    }
    else if(digit>shigh && digit<high)
    {
        shigh=digit;
    }
    num/=10;
}
if(shigh==-1)
{
    printf("all digits are same.\n");
}
else
{
    printf("second highest digit is:%d\n",shigh);
}
return shigh;
}

```

5. Print numbers within a range whose sum of digits reduces to a single digit

Write a program to print all numbers in the range, whose sum of digits is equal to 9 after reducing to single digit.

input	output
10 100	18 27 36 45 54 63 72 81 90 99
-10 100	invalid range
1000 90	invalid range

Program:

```

#include<stdio.h>
int reduceToSingleDigit(int num)
{
    int sum=0;
    while(num>0||sum>9)
    {
        if(num==0)
        {
            num=sum;
            sum=0;
        }
        sum+=num%10;
        num/=10;
    }
    return sum;
}

void printNumbersWithSumNine(int start,int end)
{
    if(start < 0 || end < 0 || start > end)
    {
        printf("Invalid range\n");
    }
}

```

```

        return;
    }
    int found=0;
    for(int i=start; i<=end;i++)
    {
        if(reduceToSingleDigit(i)==9)
        {
            printf("%d ",i);
            found=1;
        }
    }
    if(!found)
    {
        printf("No numbers found\n");
    }
    printf("\n");
}
int main()
{
    int start,end;
    printf("Enter the range(start end): ");
    if(scanf("%d %d", &start, &end)!=2)
    {
        printf("Invalid input\n");
        return 1;
    }
    printNumbersWithSumNine(start, end);
    return 0;
}

```

6. print non-repeated digits in a number.

Write a program to print non-repeated digits in a number.

example:

Input:	Output:
-----	-----
1223	3 1
233	2
3443	no non-repeated numbers
55248	8 4 2
0	0
-12254	4 5 1

Program:

```

#include<stdio.h>
void printNonRepeatedDigits(int num)
{
    if(num < 0)
    {
        num=-num;
    }
    if(num==0)
    {
        printf("0\n");
        return;
    }
}

```

```

}
int digitCount[10]={0};
int temp = num;
while(temp>0)
{
    int digit = temp%10;
    digitCount[digit]++;
    temp/=10;
}
int found=0;
temp=num;
while(temp>0)
{
    int digit=temp%10;
    if(digitCount[digit]==1)
    {
        printf("%d ",digit);
        found=1;
    }
    temp/=10;
}
if(!found)
{
    printf("no non-repeated numbers");
}
printf("\n");
}

int main()
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printNonRepeatedDigits(num);
    return 0;
}

```

VPL Arrays

1. Place all zeros to the end

Given an unsorted integer array, place all zeros to the end of the array without changing the sequence of non-zero.

Input Format

The first element of input consists of number of array elements i.e., n

The second element on-wards , input of an array.

Constraints

$1 \leq n \leq 100$

$0 \leq A[i] \leq 1000$

Output Format

Return the array after performing the task

Example:

Input:

9 1 3 0 8 12 0 4 0 7

0 9 4

Overflow

output:

1 3 8 12 4 7 0 0 0	-5 8 4 4 2 6
invalid size	101 8 3 4 6 7

invalid size
Memory

Program:

```
#include<stdio.h>
void movZerosToEnd(int arr[],int n)
{
    int j=0;
    for(int i=0;i<n;i++)
    {
        if(arr[i]!=0)
        {
            arr[j++]=arr[i];
        }
    }
    for(int i=j; i<n;i++)
    {
        arr[i]=0;
    }
}
int main()
{
    int n;

    scanf("%d",&n);
    if(n>100)
    {
        printf("Memory Overflow\n");
        return 1;
    }
    if(n<=1)
    {
        printf("Invalid size\n");
        return 1;
    }
    int arr[n];

    for(int i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
        if(arr[i]<0 || arr[i]>1000)
        {
            printf("invalid input\n");
            return 1;
        }
    }
    movZerosToEnd(arr,n);
    for(int i=0; i<n; i++)
    {
        printf("%d", arr[i]);
        if(i<n-1)
        {
            printf(" ");
        }
    }
    printf("\n");
```

```
    return 0;
}
```

2. Leaders in the array

Write a program to print all the LEADERS in the int array. An element is a leader if it is greater than all the elements to its right side. And the rightmost element is always a leader.

For example in the array {16, 17, 4, 3, 5, 2}, leaders are 17, 5 and 2.

Example:

Input: 6

16 17 4 3 5 2

Output:

17 5 2

Program:

```
#include<stdio.h>
void input(int *,int);
void output(int *,int);
int main()
{
    int a[10], n;
    scanf("%d",&n);
    input(a,n);
    output(a,n);
}
void input(int *a, int n)
{
    for(int i=0; i<n; i++)
    {
        scanf("%d",&a[i]);
    }
}
void output(int *a,int n)
{
    int cnt=0;
    if(n<=0)
        printf("Invalid size");
    else
    {
        for(int i=0; i<n;i++)
        {
            cnt=0;
            for(int j=i+1; j<n; j++)
            {
                if(a[i]<a[j])
                    cnt=1;
            }
            if(cnt==0)
            {
                printf("%d ",a[i]);
            }
        }
    }
}
```

3. Print repeated elements

Write a program to print repeated elements in an integer array. Array size must contain 20 elements size.

Note:- First data in input is number of elements and then remaining data is input for array

ex:-

input	output
5 1 2 3 4 5	no repeated elements
10 2 2 2 3 4 5 2 5 6 2	2 5
11 -2 -1 0 -1 9 5 3 2 6 8 -2	-2 -1
-3 2 3 5	invalid size
0 0	invalid size

Program:

```
#include <stdio.h>
int main() {
    int SIZE;
    // Step 1: Input the number of elements
    scanf("%d", &SIZE);
    // Input validation: check if SIZE is valid (between 1 and 20)
    if (SIZE < 1 || SIZE > 20) {
        printf("invalid size\n");
        return 0;
    }
    int arr[SIZE];
    // Step 2: Input the elements of the array
    for (int i = 0; i < SIZE; i++) {
        scanf("%d", &arr[i]);
    }
    // Step 3: Find repeated elements using a frequency array
    int foundRepeated = 0; // Flag to check if there are any repeated elements
    for (int i = 0; i < SIZE; i++) {
        int count = 0;
        // Count how many times arr[i] appears in the array
        for (int j = 0; j < SIZE; j++) {
            if (arr[i] == arr[j]) {
                count++;
            }
        }
        // If the element appears more than once and hasn't been printed yet
        if (count > 1) {
            // Check if it's already printed
            int alreadyPrinted = 0;
            for (int k = 0; k < i; k++) {
                if (arr[k] == arr[i]) {
                    alreadyPrinted = 1;
                    break;
                }
            }
            if (!alreadyPrinted) {
                printf("%d ", arr[i]);
                foundRepeated = 1;
            }
        }
    }
    // Step 4: Handle case where there are no repeated elements
    if (!foundRepeated) {
```

```

    printf("no repeated elements\n");
}
return 0;
}

```

4. Shift array to left / right

Write a program to shift an array left or right depends on choice.

Input Format

step-1:- The first element of input consists of number of array elements i.e., n

step-2:- The second element on-wards , input of an array upto n elements.

step-3:- next input is choice for left/right shift operation. [r --> right , l --> left]

step-4:- How many time need to shift elements

Constraints

1<= n <=100

0<= A[i] <=1000

Output Format

Print array elements after the task

Example:

Input:-

9
1 3 0 8 12 0 4 0 7
R
4

output:- 0 0 0 0 1 3 0 8 12

Input:-

-5
84426
l
4

output:- "No of elements are invalid"

program:

```

#include<stdio.h>
#include<stdio_ext.h>
int input(int *,int);
void shift(int *,int,char,int);
int main()
{
    int size;
    scanf("%d",&size);
    if(size<1)
    {
        printf("No of elements are invalid");
        return 0;
    }
    int a[100];
    input(a,size);
    char choice;
    __fpurge(stdin);
    scanf("%c", &choice);
    int nor;
    __fpurge(stdin);
    scanf("%d",&nor);
    __fpurge(stdin);shift(a,size,choice,nor);
}
int input(int *p,int n)
{
    int i;
    for(i=0; i<n; i++)
    {
        scanf("%d",p+i);
    }
}

```

```

    }
}
void shift(int *p,int n,char choice, int nor)
{
    while(nor>0)
    {
        int j;
        if(choice=='r')
        {
            for(j=n-1; j>=0; j--)
            {
                p[j]=p[j-1];
            }
            p[0]=0;
        }
        else if(choice=='l')
        {
            for(j=0; j<n-1; j++)
            {
                p[j]=p[j+1];
            }
            p[n-1]=0;
        }
        nor--;
    }
    for(int i=0; i<n; i++)
    {
        printf("%d ", p[i]);
    }
}

```

5. Rotate array left/right

Write a program to rotate an array left or right depends on choice.

Input Format

step-1:- The first element of input consists of number of array elements i.e., n

step-2:- The second element on-wards , input of an array upto n elements.

step-3:- next input is choice for left/right shift operation. [r --> right , l --> left]

step-4:- How many time need to shift elements

Constraints

1<= n <=100

0<= A[i] <=1000

Output Format

Print array elements after the task

Example:

Input:-

9
1 3 0 8 12 0 4 0 7
r
4

output:- 0 4 0 7 1 3 0 8 12

Input:-

-5
8 4 4 2 6
l
4

output:- "No of elements are invalid"

program:

```
#include<stdio.h>
```

```
#include<stdio_ext.h>
```

```
int input(int*, int);
```

```

int rotate(int *,int);
int main()
{
    int size;
    scanf("%d", &size);
    if(size<1)
    {
        printf("No of elements are invalid");
        return 0;
    }
    int arr[100];
    input(arr,size);
    rotate(arr,size);
}
int input(int *p, int n)
{
    int i;
    for(i=0; i<n;i++)
    {
        scanf("%d",p+i);
    }
}
int rotate(int *p, int n)
{
    char choice;
    __fpurge(stdin);
    scanf("%c",&choice);
    int nor;
    __fpurge(stdin);
    scanf("%d",&nor);
    int j, temp;
    while(nor>0)
    {
        if(choice=='r')
        {
            temp=p[n-1];
            for(j=n-1;j>0;j--)
            {
                p[j]=p[j-1];
            }
            p[0]=temp;
        }
        if(choice=='l')
        {
            temp=p[0];
            for(j=0;j<n-1;j++)
            {
                p[j]=p[j+1];
            }
            p[n-1]=temp;
        }
        nor--;
    }
}

```

```

int i;
for(i=0;i<n;i++)
{
    printf("%d ",p[i]);
}
}

```

6. Maximum sum of any contiguous subarray

Problem Statement

Given an array of positive numbers and a positive number 'k,' find the maximum sum of any contiguous subarray of size 'k'.

Example 1:

Input: [2, 1, 5, 1, 3, 2], k=3

Output: 9

Explanation: Subarray with maximum sum is [5, 1, 3].

Example 2:

Input: [2, 3, 4, 1, 5], k=2

Output: 7

Explanation: Subarray with maximum sum is [3, 4].

Input Format

step-1:- The first element of input consists of number of array elements i.e., n

step-2:- The second element on-wards , input of an array upto n elements.

step-3:- Next input is size/no.of elements of sub array i.e., k

step-4:- Find maximum sum of sub array in main array

Constraints

$1 \leq n \leq 100$

$0 \leq A[i] \leq 1000$

Output Format

Print sum-of that sub array

Program:

```
#include <stdio.h>
```

```

int main() {
    int n, k;
    // Step 1: Input the number of elements in the array
    printf("Enter number of elements in the array: ");
    scanf("%d", &n);
    // Edge case: If n is invalid (<= 0 or > 100), print error and exit
    if (n <= 0 || n > 100) {
        printf("invalid main array size\n");
        return 0;
    }
    // Step 2: Input the array elements
    int arr[n];
    printf("Enter the elements of the array: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    // Step 3: Input the size of the subarray k
    printf("Enter the size of the subarray (k): ");
    scanf("%d", &k);
    // Edge case: If k is greater than n or k <= 0, print error and exit
    if (k > n || k <= 0) {
        printf("invalid sub array size\n");
        return 0;
    }
}

```

```

}
// Step 4: Calculate the maximum sum of a subarray of size k using the sliding window technique
int max_sum = 0, current_sum = 0;
// Compute the sum of the first 'k' elements
for (int i = 0; i < k; i++) {
    current_sum += arr[i];
}
max_sum = current_sum;
// Now, slide the window
for (int i = k; i < n; i++) {
    current_sum += arr[i] - arr[i - k]; // Slide the window by adding the next element and removing the old element
    if (current_sum > max_sum) {
        max_sum = current_sum;
    }
}
// Step 5: Output the result
printf("Maximum sum of a subarray of size %d is: %d\n", k, max_sum);
return 0;
}

```

7. Print non-repeated elements

Write a program to print non-repeated elements in a array.

int arr[size]; -----> 20<= size <=50.

Note:- first element in input is no.of elements and then rest of elements are input data

examples:-

input	output	
0 5	invalid size	-3 2 5 7 invalid size
5 1 2 3 4 5	1 2 3 4 5	55 3 4 6 7 9 0 Memory Overflow
6 8 2 4 8 7 9	2 4 7 9	3 9 9 9 no non-repeated elements

Program:

```

#include<stdio.h>
int main()
{
    int n,i,j,flag=0,s=0;
    int arr[20];
    scanf("%d",&n);
    for(i=0;i<n;i++)
        scanf("%d",(arr+i));
    if(n>20)
    {
        printf("memory overflow");
        return 0;
    }
    if(n<=0)
    {
        printf("invalid size");
        return 0;
    }
    for(i=0;i<n;i++)
    {
        for(j=0,flag=0;j<n;j++)
            if(arr[i]==arr[j] && i!=j)
            {
                flag=1;
            }
    }
}

```

VPL MACRO

1. reverse all bits in a given integer (complete it by using MACROS)

Note: Reverse bits operation should perform on same variable

```
// define MACROS REV_CON , REV_LOG to get result
```

input	output
-----	-----
65	10000010000000000000000000000000-2113929216
-3223	10010110110011111111111111111111-1764753409
0	00000000000000000000000000000000

```
#include <stdio.h>
```

```
#define REV_CON(data, l, r) (((data >> l) & 1) != ((data >> r) & 1))
```

```
#define REV_LOG(data, l, r)
```

```
int main() {
    int data, bit, l, r, i;
    // Read the input data
    scanf("%d", &data);
    // Swap bits starting from the outermost and move towards the center
    for (l = 31, r = 0; l > r; l--, r++)
    {
        if (REV_CON(data, l, r))
        {
            REV_LOG(data, l, r);
        }
    }
    // Print the binary representation of the reversed number
    for (i = 31; i >= 0; i--)
    {
```

```

    printf("%d ", (data >> i) & 1);
}
// Print the integer value of the reversed data
printf(" %d\n", data);
return 0;
}

```

2. count pair of set bits (complete it by using MACROS)

write a program to print [pair of set bits count](#) in a given integer?

example:-

input	output		
-1	16	-3	15
-5234	11	65	0
0	0	85363	3

Program:

```

#include<stdio.h>
#define STATUS_BIT(WORD,BIT)((WORD>>BIT)&1)
int main()
{
    int n,bit,cnt=0;
    scanf("%d",&n);
    for(bit=31; bit>0;bit--)
    {
        if(STATUS_BIT(n,bit)&& STATUS_BIT(n,bit-1))

        {
            cnt++;
            bit--;
        }
    }
    printf("%d",cnt);
    return 0;
}

```

3. count pair of set bits (complete it by using MACROS)

write a program to print [pair of set bits count](#) in a given integer?

example:-

input	output		
-1	16	-3	15
-5234	11	65	0
0	0	85363	3

Program:

```

#include<stdio.h>
#define STATUS_BIT(WORD,BIT)((WORD>>BIT)&1)
int main()
{
    int n,bit,cnt=0;
    scanf("%d",&n);
    for(bit=31; bit>0;bit--)
    {
        if(STATUS_BIT(n,bit)&& STATUS_BIT(n,bit-1))

        {
            cnt++;
            bit--;
        }
    }
}

```

```

    }
}
printf("%d",cnt);
return 0;
}

```

VPL Strings

Convert all lower case chars into upper case and vice versa

examples:-

input	output
"vector"	VECTOR
"12ve NZl\$"	12 VE nzL\$
" "	ERROR

Program:

```

#include<stdio.h>
#include<ctype.h>
#include<string.h>
#include<stdbool.h>
int main()
{
    char s[100];
    int len;
    int has_alpha=0;
    printf("enter a string:");
    fgets(s, sizeof(s),stdin);
    len=strlen(s);
    if(len>0 && s[len-1]=='\n')
    {
        s[len-1]='\0';
    }
    if(strcmp(s,"")==0 || strcmp(s," ")==0)
    {
        printf("ERROR\n");
        return 0;
    }
    bool is_empty_or_whitespace=true;
    for(int i=0;s[i]!='\0';i++)
    {
        if(!isspace(s[i]))
        {
            is_empty_or_whitespace=false;
            break;
        }
    }
    if(is_empty_or_whitespace)
    {
        printf("ERROR\n");
        return 0;
    }
    for(int i=0; s[i]!='\0';i++)
    {
        if(isalpha(s[i]))
        {

```

```

        has_alpha = 1;
        if(islower(s[i]))
        {
            s[i]=toupper(s[i]);
        }
        else if(isupper(s[i]))
        {
            s[i]=tolower(s[i]);
        }

    }
}
if(!has_alpha)
{
    printf("%s\n",s);
}
else
{
    printf("Resultant string is %s\n",s);
}
return 0;
}

```

Print number of times a given character repeated

write a program to print number of times a given character repeated in a given string.

note: should be case sensitive

Input:

```

"embedded" 'e'
"vector india hyderabad" 'v'
"vector india hyderabad" 'H'
"hello hyderabad" ' '

```

Output:

```

3
1
not found
5

```

Program:

```

#include<stdio.h>
int main()
{
    char input[100];
    char ch;
    int count=0;
    printf("enter a string: ");
    fgets(input,sizeof(input),stdin);
    printf("enter the character to count: ");
    scanf("%c",&ch);
    for(int i=0; input[i]!='\0';i++)
    {
        if(input[i]==ch)
        {
            count++;
        }
    }
    if(count>0)
    {
        printf("%d\n",count);
    }
}

```

```

    }
    else
    {
        printf("not found\n");
    }
    return 0;
}

```

Hide the pair of vowels

write a program to hide the pair of vowels in given string.

Note:- No case-sensitive

examples:-

input

vector india

vector in hyd

ind has bEen developing

for me its okay

output

vector ind**

there is no pair of vowels to hide

ind has b**n developing

there is no pair of vowels to hide

program:

```

#include<stdio.h>
#include<string.h>
#include<ctype.h>
int is_vowel(char c)
{
    c=tolower(c);
    return(c=='a' || c=='e' || c=='i' || c=='o' || c=='u');
}
int is_valid_input(const char *str)
{
    while(*str)
    {
        if(!isalpha(*str) && !isspace(*str))
        {
            return 0;
        }
        str++;
    }
    return 1;
}
int main()
{
    char input[100];
    char output[100];
    int found_pair=0;
    printf("Enter a string: ");
    fgets(input, sizeof(input), stdin);
    size_t len = strlen(input);
    if(len>0 && input[len-1]=='\n')
    {
        input[len-1]='\0';
    }
    if(!is_valid_input(input))
    {
        printf("invalid input\n");
        return 0;
    }
}

```

```

}
int j=0;
for(int i=0; input[i]!='\0';i++)
{
    if(is_vowel(input[i]) && is_vowel(input[i+1]))
    {
        output[j++]='*';
        output[j++]='*';
        found_pair=1;
        i++;
    }
    else
    {
        output[j++]=input[i];
    }
}
output[j]='\0';
if(found_pair)
{
    printf("%s\n",output);
}
else
{
    printf("there is no pair of vowels to hide\n");
}
return 0;
}

```

Anagram

WAP to verify two given strings are anagrams or not.

Example:

Input1:

silent

listen

Input2:

Cat

Act

Input3:

cat

act

Output: Anagram string.

Output: Not an anagram string.

Output: Anagram string.

Program:

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
void sorting(char *p1,char *p2)
{
    int i,j;
    int flag, n1=strlen(p1), n2=strlen(p2);
    for(i=n1-1; i>0;i--)
    {
        flag=1;
        for(j=0;j<i;i++)
        {

```

```

        if(p1[j]>p1[j+1])
        {
            p1[j]^=p1[j+1]^=p1[j]^=p1[j+1];
            flag=0;
        }
    }
    if(flag==1)
    {
        break;
    }
}
for(i=n2-1;i>0;i--)
{
    flag=1;
    for(j=0;j<i;j++)
    {
        if(p2[j]>p2[j+1])
        {
            p2[j]^=p2[j+1]^=p2[j]^=p2[j+1];
            flag=0;
        }
    }
    if(flag==1)
    {
        break;
    }
}
}
int main()
{
    char s1[100],s2[100];
    fgets(s1,100,stdin);
    if(s1[strlen(s1)-1]=='\n')
    {
        s1[strlen(s1)-1]='\0';
    }
    fgets(s2,100,stdin);
    s2[strlen(s2)-1]='\0';
    sorting(s1,s2);
    if(strcmp(s1,s2)==0)
    {
        printf("Anagram strings");
    }
    else
    {
        printf("Not an anagram strings");
    }
    return 0;
}

```

Remove Extra spaces in a string

write a program to count the no. of palindromes in the given string

Input:

"vector india hyderabad"

Output:

"vector india hyderabad"

```
" vector india hyderabad "  
"
```

```
"vector india hyderabad"  
empty string
```

Program:

Print a word which is start with vowel and end with vowel

Write a program to print a word which is start with vowel and end with vowel in a given string.

input

output

```
-----  
india is my country          india  
back to india once          india once  
i m using my brain once     i once  
all the best                 not found
```

program:

```
#include<stdio.h>  
#include<string.h>  
#include<ctype.h>  
  
int isVowel(char c)  
{  
    c = tolower(c);  
    return(c=='a' || c=='e' || c=='i' || c=='o' || c=='u');  
}  
  
int main()  
{  
    char str[200];  
    char *token;  
    int found=0;  
  
    printf("Enter a string: ");  
    if(fgets(str, sizeof(str), stdin)==NULL || str[0]=='\n')  
    {  
        printf("empty string\n");  
        return 0;  
    }  
    str[strcspn(str, "\n")]='\0';  
    token = strtok(str, " ");  
    if(token==NULL)  
    {  
        printf("empty string\n");  
        return 0;  
    }  
    while(token!=NULL)  
    {  
        if(isVowel(token[0]) && isVowel(token[strlen(token)-1]))  
        {  
            printf("%s ", token);  
            found=1;  
        }  
        token = strtok(NULL, " ");  
    }  
    if(found)  
        printf("\n");  
}
```

```

    }
    token= strtok(NULL, " ");
}
if(!found)
{
    printf("there is no words to print\n");
}
return 0;
}

```

reverse all occurrences of sub string in main string

Write a program to reverse all occurrences of sub string in main string

Example:-

input:- s1= "this is vector" , s2= "is"		input:- s1="this is vector" , s2="india"
output:- s1= "thsi si vector"		output:- "sub string not found"

program:

```

#include<stdio.h>
#include<string.h>
#include<stdio_ext.h>
void reverse(const char *, char *);
int main()
{
    char str[30], sub[10], rev[10];
    char *ptr;
    char *main=str;

    scanf("%[^\\n]s", str);
    __fpurge(stdin);
    scanf("%[^\\n]s", sub);
    __fpurge(stdin);

    if((strstr(str, sub)==NULL))
    {
        printf("sub string not found");
        return 0;
    }
    reverse(sub, rev);

    while(ptr=strstr(main,sub))
    {
        strncpy(ptr, rev, strlen(rev));
        main=ptr+strlen(rev);
    }
    puts(str);
    return 0;
}
void reverse(const char *sub, char *rev)
{
    int i,j;
    int len=strlen(sub);
    for(i=0, j=len-1; i<len; i++,j--)
        rev[i]=sub[j];
}

```

Replace all occurrences of sub string in main string with opposite case of every alphabe

Write a program to replace all occurrences of sub string in main string with opposite case of every alphabet.

Example:-

input:- s1= "thABis is vector" , s2= "AB"
output:- s1= "thabis is vector"

input:- s1= "this is abc vector" , s2= "abc"
output:- s1= "this is ABC vector"

input:- s1= "this is AbC vector" , s2= "AbC"
output:- s1= "this is aBc vector"

input:- s1="this is vector" , s2="india"
output:- "sub string not found"

program:

```
#include<stdio.h>
#include<string.h>
#include<stdio_ext.h>
void oppCase(const char*,char *);
int main()
{
    char str[30], sub[10], opp[10], *ptr, *main;
    main=str;

    scanf("%[^\\n]s",str);
    __fpurge(stdin);
    scanf("%[^\\n]s",sub);
    __fpurge(stdin);

    if(strstr(str, sub)==NULL)
    {
        printf("sub string not found");
        return 0;
    }
    oppCase(sub,opp);
    while(ptr=strstr(main,sub))
    {
        strncpy(ptr,opp,strlen(opp));
        main=ptr+strlen(opp);
    }
    puts(str);
    return 0;
}
void oppCase(const char *sub,char *p)
{
    while(*sub)
    {
        *p=(*sub)^(1<<5);
        sub++;
        p++;
    }
}
```

count the no.of palindromes in the given string

Input:

Output:

"racecar drivers enjoy radar."

2

"madam, in Eden, I am bob."

3

"elle is a teacher, as I am."

3

```
"vector india hyderabad"  
"
```

no palindromes in the given string
empty string

Program:

```
#include<stdio.h>  
#include<string.h>  
#include<ctype.h>  
int main()  
{  
    char str[100];  
    int i,l,count=0,empty=0;  
    scanf("%s",str);  
    //empty string  
    for(i=0;str[i];i++)  
    {  
        if(str[i]!=' ')  
        {  
            empty=1;  
            break;  
        }  
    }  
    if(empty==0)  
    {  
        printf("empty string");  
        return 0;  
    }  
    //logic  
    char *ptr= strtok(str, " ");  
    while(ptr != NULL)  
    {  
        l=strlen(ptr);  
        //lower case  
        for(i=0;i<l;i++)  
        {  
            ptr[i]=tolower(ptr[i]);  
        }  
        //palindrom  
        for(i=0;i < l/2;i++)  
        {  
            if(ptr[i]!=ptr[l-1-i])  
                break;  
        }  
        if(i==l/2)  
            count++;  
        ptr=strtok(NULL, " ");  
    }  
    if(count==0)  
        printf("no palindromes in the given string");  
    else  
        printf("%d",count);  
    return 0;  
}
```

check whether the user input is integer or float type.

Write a program to check whether the user input is integer or float data type.

note:- read input in string form

Example:

input: 82 **Output:** Integer number

input: 80.83 **Output:** Floating number

input: 8&0.83 **Output:** invalid input

program:

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#include<ctype.h>
```

```
int isInteger(const char *str)
```

```
{
    for(int i=0; str[i]!='\0'; i++)
    {
        if(!isdigit(str[i]))
        {
            return 0;
        }
    }
    return 1;
}
```

```
int isFloat(const char *str)
```

```
{
    int dotCount = 0;
    for (int i=0; str[i]!='\0'; i++)
    {
        if(str[i]=='.')
        {
            dotCount++;
            if(dotCount>1)
            {
                return 0;
            }
        }
        else if(!isdigit(str[i]))
        {
            return 0;
        }
    }
    return dotCount == 1;
}
```

```
int main()
```

```
{
    char input[100];
    printf("Enter a number: ");
    //scanf("%s", input);
    fgets(input, sizeof(input), stdin);
    size_t len = strlen(input);
    if(len>0 && input[len-1]=='\n')
    {
        input[len-1]='\0';
    }
}
```

```

}
if(isInteger(input))
{
    printf("Integer number\n");
}
else if(isFloat(input))
{
    printf("Float number\n");
}
else
{
    printf("Invalid input\n");
}
return 0;
}

```

Implement binary to int conversion

write a program to implement binary to int conversion

note:- input is string form

ex:-

input	output
-----	-----
001111	15
10000000	128
01111111	127
00000000	0
101010a00	invalid input

Program:

```

#include<stdio.h>
#include<string.h>

```

```

int inValidInput(char *);
int btoi(char *);

```

```

int main()
{
    char str[20];
    int len;
    scanf("%s",str);
    if(inValidInput(str))
    {
        printf("invalid input");
        return 0;
    }
    printf("%d", btoi(str));
    return 0;
}

int inValidInput(char *s)
{
    while(*s)
    {
        if(*s!='0' && *s!='1')
            return 1;
        s++;
    }
}

```

```

    }
    return 0;
}
int btoi(char *s)
{
    char *temp;
    int res=0;
    int p=1;
    temp=s+(strlen(s)-1);
    while(temp>=s)
    {
        res=res+(p*((*temp)-48));
        p*=2;
        temp--;
    }
    return res;
}

```

Integer to String/Ascii Conversion

--> Implement a user-defined **itoa** function.

Notes:

- You should take an integer type variable to collect an integer value from the user input. Then convert the input integer number to a string.
- You should take a character array and store the converted integer string to that array.

Example:

<u>Input:</u>	----->	<u>Output</u>
12563	----->	"12563"
-2345	----->	"-2345"

Program:

```

#include<stdio.h>
#include<stdlib.h>

void my_itoa(char *,int,int);
int main()
{
    int num, temp,i,cnt=0;
    char *str[20];

    scanf("%d",&num);
    if(num==0)
    {
        printf("0");
        return 0;
    }
    temp=abs(num);
    while(temp)
    {
        cnt++;
        temp/=10;
    }

    my_itoa(str,cnt,num);
}

```

```

printf("%s", str);

return 0;
}
void my_itoa(char *p,int n, int temp)
{
    int temp2;
    if(temp<0)
    {
        *p='-';
        p++;
    }
    p=p+(n-1);
    temp2=abs(temp);
    while(temp2)
    {
        *p=temp2%10+48;
        p--;
        temp2/=10;
    }
}

```

Implement ATOI user defined function

Write a program to implement user defined function of atoi

Ex:-

input=12345

output=12345

input=12a45

output=invalid

input=1234.5

output=invalid

program:

```

#include<stdio.h>
#include<string.h>
#include<ctype.h>
int my_atoi(char *);
int checkValid(char*);
int main()
{
    char str[20];
    printf("input=");
    fgets(str, 20,stdin);
    if(str[strlen(str)-1]==10)
        str[strlen(str)-1]=0;
    if(checkValid(str))
    {
        printf("invalid");
        return 0;
    }
    printf("%d", my_atoi(str));
    return 0;
}
int checkValid(char *p)
{
    while(*p)
    {
        if(*p=='-')
            ++p;
    }
}

```

```

        if(!(isdigit(*p)))
            return 0;
    }
}
int my_atoi(char *p)
{
    int r=0, f=1;
    if(*p=='-')
    {
        f=-1;
        p++;
    }
    while(*p)
    {
        r=r*10+(*p-48);
        p++;
    }
    return r*f;
}

```

Implement atof user-defined function

implement atof user-defined function

examples:-

input	output		
-----	-----		
"123.23"	123.230000	"123."	ERROR
"123a.43"	ERROR	"-41.33"	-41.330000
"343"	ERROR	"5424.0"	5424.000000
"123.13345a"	ERROR		

Program:

```

#include<stdio.h>
#include<string.h>
#include<ctype.h>
int isValidInput(char *);
float my_atof(char *);
int main()
{
    char s1[30];
    float f;
    fgets(s1, 30, stdin);
    if(s1[strlen(s1)-1]=='\n')
        s1[strlen(s1)-1]=0;

    if(isValidInput(s1))
    {
        printf("ERROR");
        return 0;
    }
    f=my_atof(s1);
    if(f==0)
        printf("%0.3f",f);
    else
        printf("%0.3f",f);
}

```

```

    return 0;
}
int isValidInput(char *f)
{
    int s=0;
    if(*f=='-')
        f++;
    while(*f)
    {
        if(!(isdigit(*f)) && (*f)!='.'))
            return 1;
        if(*f=='.')
        {
            s=1;
            if(*(f+1)==0)
                return 1;
        }
        f++;
    }
    if(s==0)
        return 1;
    return 0;
}

```

```

float my_atof(char *p)
{
    int cnt=0,s=1,b;
    float floa=0, deci=0, res;
    float a=10;
    if(*p=='-')
    {
        s=-1;
        p++;
    }
    while(*p)
    {
        if(*p=='.')
        {
            p++;
            break;
        }
        deci=deci*10+(*p)-48;
        cnt++;
        p++;
    }
    while(*p)
    {
        floa=floa+(((p)-48)/a);
        a*=10;
        p++;
    }
    res=(deci+floa)*s;
    return res;
}

```

Implement FTOA user defined function

Write a C function ftoa() that converts a given floating-point number or a double to a string.

Note :- Print only 4 digits in fractional part

examples:-

input	output
-----	-----
123.23	123.2300
123.12345678"	123.1234

Program:

```
#include<stdio.h>
#include<string.h>
char arr[10];
void ftoa(float f)
{
    int a,i,cnt=0,j,k,b,s=0,flag=0;
    char temp;
    a=f;
    i=0;
    if(f==0.000000)
    {
        arr[i++]='0';
        arr[i++]='.';
        for(j=0;j<4;j++)
            arr[i++]='0';
        return;
    }
    i=0;
    if(f<0)
    {
        arr[i++]='_';
        a=-a;
        f=-f;
        flag=1;
    }
    f=f-a;
    if(a==0) arr[i++]='0';
    while(a)
    {
        arr[i++]=(a%10)+48;
        a/=10;
    }
    j=0;
    k=strlen(arr)-1;
    if(flag==1)
    {
        j=1;
    }
    while(j<k)
    {
        temp=arr[j];
        arr[j]=arr[k];
        arr[k]=temp;
        j++;
    }
}
```

```

        k--;
    }
    arr[i++]='.';
    f*=10;
    cnt=0;
    while(b=f)
    {
        a=f;
        arr[i++]=a+48;
        f=f-a;
        f*=10;
        cnt++;
        if(cnt>=4)
            break;
    }
    if(cnt<4)
        for(cnt;cnt<4;cnt++)
            arr[i++]='0';
}
int main()
{
    float f;
    int a,i;
    scanf("%f",&f);
    floa(f);
    for(i=0;i<strlen(arr);i++)
    {
        printf("%c",arr[i]);
    }
    return 0;
}

```

Palindrome part-1

Write a program to check given string is palindrome or not.

ex:-

input	output
-----	-----
madam	palindrome
noon	palindrome
kill	not palindrome

program:

```

#include<stdio.h>
#include<string.h>
int main()
{
    char str[20], *left, *right;
    int flag=0;
    fgets(str, 20,stdin);
    if(strlen(str)-1==10)
        str[strlen(str)-1]=0;
    left=str;
    right=str+(strlen(str)-1);
    while(left<right)

```

```

{
    if(*left!=*right)
    {
        flag=1;
        break;
    }
    left++;
    right--;
}
if(flag==1)
printf("not palindrome");
else
printf("palindrome");
return 0;
}

```

Test palindrome by removing one char ---> Palindrome part-2

Given a string of lowercase letters in the range `ascii[a-z]`, determine the index of one character that can be removed to make the string a palindrome. There may be more than one solution, but any will do.

If the word is already a palindrome return -1 and if there is no solution, print "can not be a palindrome" . Otherwise, return the index of a character to remove.

Input format

string s: a string to analyze

Return value

The index of the character to remove , if there is a solution

-1 , if it is already a palindrome

print "can not be a palindrome" if no solution.

Constraints

All characters are in the range `ascii[a-z]` lowercase only.

Explanation

Query 1: "aaab"

Removing 'b' at index 3 results in a palindrome, so return 3.

Query 2: "baa"

Removing 'b' at index 0 results in a palindrome, so return 0.

Program:

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#include<stdbool.h>
```

```
bool isPalindrome(char *s, int left, int right)
```

```

{
    while(left<right)
    {
        if(s[left]!=s[right])
        {
            return false;
        }
        left++;
        right--;
    }
    return true;
}

```

```
int palindromeIndex(char *s)
```

```

{
    int left = 0;

```

```

int right = strlen(s)-1;
while(left<right)
{
    if(s[left]!=s[right])
    {
        if(isPalindrome(s, left+1, right))
        {
            return left;
        }
        else if(isPalindrome(s,left,right-1))
        {
            return right;
        }
        else
        {
            //printf("can not be a palindrome\n");
            return -2;
        }
    }
    left++;
    right--;
}
return -1;
}

int main()
{
    char s[100];
    printf("Enter a string: ");
    scanf("%s", s);
    int result = palindromeIndex(s);
    if(result==-1)
    {
        printf("already a palindrome\n");
    }
    else if(result!=-2)
    {
        printf("%d\n",result);
    }
    else{
        printf("can not be a palindrome\n");
    }
    return 0;
}

```

Convert all numeric characters to a word

Write a program to convert all numeric characters to a word from a given string.

Example:

Input: Hello 24India

Output: Hello TwoFourIndia

Input: Programming practice

Output: Numeric character not found.

Input: Virtual34 Prog56ramming

Output: VirtualThreeFour ProgFiveSixramming

Program:

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#include<ctype.h>
```

```

void convertDigitToWord(char* str)
{
    int len = strlen(str);
    int foundNumeric=0;
    char result[1000]="";
    char *digitWords[] = {"Zero", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine"};
    for(int i=0;i<len; i++)
    {
        if(isdigit(str[i]))
        {
            foundNumeric = 1;
            char word[10];
            int index = str[i]-'0';
            strcat(result,digitWords[index]);
        }
        else
        {
            strncat(result,&str[i],1);
        }
    }
    if(!foundNumeric)
    {
        printf("Numeric character not found.\n");
    }
    else
    {
        printf("%s\n", result);
    }
}

int main()
{
    char str[100];
    printf("Enter a string: ");
    fgets(str,sizeof(str), stdin);
    if(str[strlen(str)-1]=='\n')
    {
        str[strlen(str)-1]='\0';
    }
    convertDigitToWord(str);
    return 0;
}

```

Insert STRING_2 at specific location in STRING_1

WAP to insert STRING_2 at specific location in STRING_1.

By using **user defined STRCPY** function.

for example:-

input1:- s1="vector" , s2="XYZ" , n=3.

output1:- "vecXYZtor" (STRING_1 data)

input2:- s1="vector" , s2="XYZ" , n=9.

output2:- invalid location

program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
// User-defined strcpy function
void my_strcpy(char *dest, const char *src) {
    while (*src) {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0';
}

int main() {
    char s1[100], s2[50], result[150];
    int n;
    // Input STRING_1, STRING_2, and position
    printf("Enter STRING_1: ");
    scanf("%s", s1);
    printf("Enter STRING_2: ");
    scanf("%s", s2);
    printf("Enter the position (n): ");
    scanf("%d", &n);
    int len1 = strlen(s1);
    int len2 = strlen(s2);
    // Check if position is valid
    if (n < 0 || n > len1) {
        printf("Invalid location\n");
        return 0;
    }
    // Build the result string
    int i = 0;
    // Copy the part of STRING_1 before the position
    for (i = 0; i < n; i++) {
        result[i] = s1[i];
    }
    // Copy STRING_2 into the result
    my_strcpy(result + i, s2);
    i += len2;
    // Copy the remaining part of STRING_1 after the position
    my_strcpy(result + i, s1 + n);
    // Print the result
    printf("%s\n", result);
    return 0;
}
```

Convert first and last char of every word

write a program to convert first and last char's to upper case in all words in given string

note:- all elements must contain alphabets only

examples:-

input	output
-----	-----
abc kmv rgrD	AbC KmV RgrD
34 KNkjkKA fefk	ERROR
VECTOR INDIA	VECTOR INDIA

Program:

```
#include<stdio.h>
#include<string.h>
```

```

#include<ctype.h>
void processString(char *str)
{
    int len=strlen(str);
    int isError=0;
    for(int i=0; i<len; i++)
    {
        if(str[i]==' ') continue;
        if(!isalpha(str[i]))
        {
            isError=1;
            break;
        }
        int start=i;
        while(i<len && str[i]!=' ')
            i++;
        int end = i-1;
        str[start] = toupper(str[start]);
        str[end] = toupper(str[end]);
    }
    if(isError)
    {
        printf("ERROR\n");
        //return;
    }else{
        printf("%s\n", str);
    }
}
int main()
{
    char input[100];
    printf("Enter a string: ");
    fgets(input, sizeof(input),stdin);
    size_t len =strlen(input);
    if(len>0 && input[len-1]=='\n')
    {
        input[len-1]='\0';
    }
    processString(input);
    return 0;
}

```

Convert the given number into words (PART-1)

Write a program to convert the given number into words.

Input:	Output:		
1	one	12	one two
-521	minus five two one	1000	one zero zero zero
9999	nine nine nine nine	0	zero

Program:

```

#include<stdio.h>
#include<string.h>
void numberToWords(char *num)
{
    char *digitToWord[]={ "zero","one","two", "three", "four", "five", "six","seven","eight","nine"};

```

```

for(int i=0; num[i]!='\0';i++)
{
    if(num[i]=='-')
    {
        printf("minus ");

    }
    else if(num[i]>='0' && num[i]<='9')
    {
        printf("%s",digitToWord[num[i]-'0']);

        if(num[i+1]!='\0')
        {
            printf(" ");
        }
    }
}
printf("\n");
}
int main()
{
    char input[100];
    printf("Enter a number: ");
    scanf("%s", input);
    numberToWords(input);
    return 0;
}

```